

Workshop: Functors, Lambdas, File I/O, and Error Handling

In this workshop, you will build a small modular C++ program that performs operations on vectors of integers. You will create a reusable module for vector processing, use **STL algorithms** together with **functors** and **lambda expressions**, and use **file I/O**, paired with **error handling** to save and retrieve data from files without resulting in crashes.

Part 1 — Understanding the Main Program

You are provided with the following `main.cpp` file. Your task is to implement a module called `vectorops` so that this program compiles and runs **without modifying the main file**.

Part 2 — Creating the Module

Create the following files:

```
vectorops.h  
vectorops.cpp
```

All functions must be placed inside the namespace:

```
namespace vectorops  
{  
}
```

Part 3 — Required Functions

You must implement the following functions.

1. Display Function

Prototype:

```
void display(const std::vector<int>& v);
```

Requirements:

- Use `std::for_each`
- Use a **lambda expression**

- Print values separated by spaces
 - End with `std::endl`
-

2. Sort Function

Prototype:

```
void sortVector(std::vector<int>& v, bool ascending);
```

Requirements:

- Use `std::sort`
 - Implement a **functor** to be used by `std::sort` for comparison. This functor should have a property called `ascending`
 - If `ascending == true` the functor should return `true` if the second number is larger
 - If `ascending == false` the functor should return `true` if the first number is larger
-

3. Add To Each Element

Prototype:

```
void addToEach(std::vector<int>& v, int value);
```

Requirements:

- Use `std::transform`
 - Implement a **functor** to be used by `std::transform`. This functor should have a property called `amount`
 - The value of `amount` is set by using `value`
 - This functor, applied to `std::transform` should result in your original vector being updated by adding `value` to each of its elements
-

4. Count Elements Greater Than

Prototype:

```
int countGreaterThan(const std::vector<int>& v, int threshold);
```

Requirements:

- Use `std::count_if`
 - Use a **lambda expression** to compare each element to the provided threshold
-

5. Save Vector to File

Prototype:

```
bool saveToFile(const std::vector<int>& v, const std::string& filename);
```

Requirements:

- Use `std::ofstream`
- Use error handling to avoid crashes
- Write integers separated by spaces
- Return `false` if file cannot be opened
- Return `true` on success

Example file:

6. Load Vector from File

Prototype:

```
std::vector<int> loadFromFile(const std::string& filename);
```

Requirements:

- Use `std::ifstream`
 - Use error handling to avoid crashes
 - Read integers from file
 - Store them in a vector
 - Return the vector
-

Part 5 — Expected Program Output

When the program runs, the output should be similar to:

```
Original vector: 9 3 12 1 7 15 4
Sorted ascending: 1 3 4 7 9 12 15
Sorted descending: 15 12 9 7 4 3 1
After adding 5 to each element: 20 17 14 12 9 8 6
Count greater than 10: 4
Vector saved to numbers.txt
Loaded vector: 20 17 14 12 9 8 6
```

Make sure that the program does not crash if the file cannot be loaded.

Deliverables

Submit the following files:

```
vectorops.h  
vectorops.cpp
```